

Lab # 04: Decision Tree Classifier

Student Information

- **Name:** Abdul Hayy Khan
- **Roll No:** 24F-AI-051

Task 1: Load Dataset

- Use a dataset such as Iris dataset (from sklearn) or load any CSV file.

```
# importing libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

# loading iris dataset
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target
df["species"] = df["target"].map(dict(enumerate(iris.target_names)))
display(df.head())
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	
0.2				
1	4.9	3.0	1.4	
0.2				
2	4.7	3.2	1.3	
0.2				
3	4.6	3.1	1.5	
0.2				
4	5.0	3.6	1.4	
0.2				

	target	species
0	0	setosa
1	0	setosa
2	0	setosa

```
3      0  setosa
4      0  setosa
```

Task 2: Exploratory Data Analysis (EDA)

- Display dataset information, shape, and statistical summary.
- Plot distributions of features (histograms / pairplot).
- Visualize correlations

```
# shape of dataset
print("Dataset shape:", df.shape)
```

Dataset shape: (150, 6)

```
# dataset information
print("Dataset info:", df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   sepal length (cm)      150 non-null   float64
1   sepal width (cm)       150 non-null   float64
2   petal length (cm)      150 non-null   float64
3   petal width (cm)       150 non-null   float64
4   target                 150 non-null   int64
5   species                150 non-null   object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
Dataset info: None
```

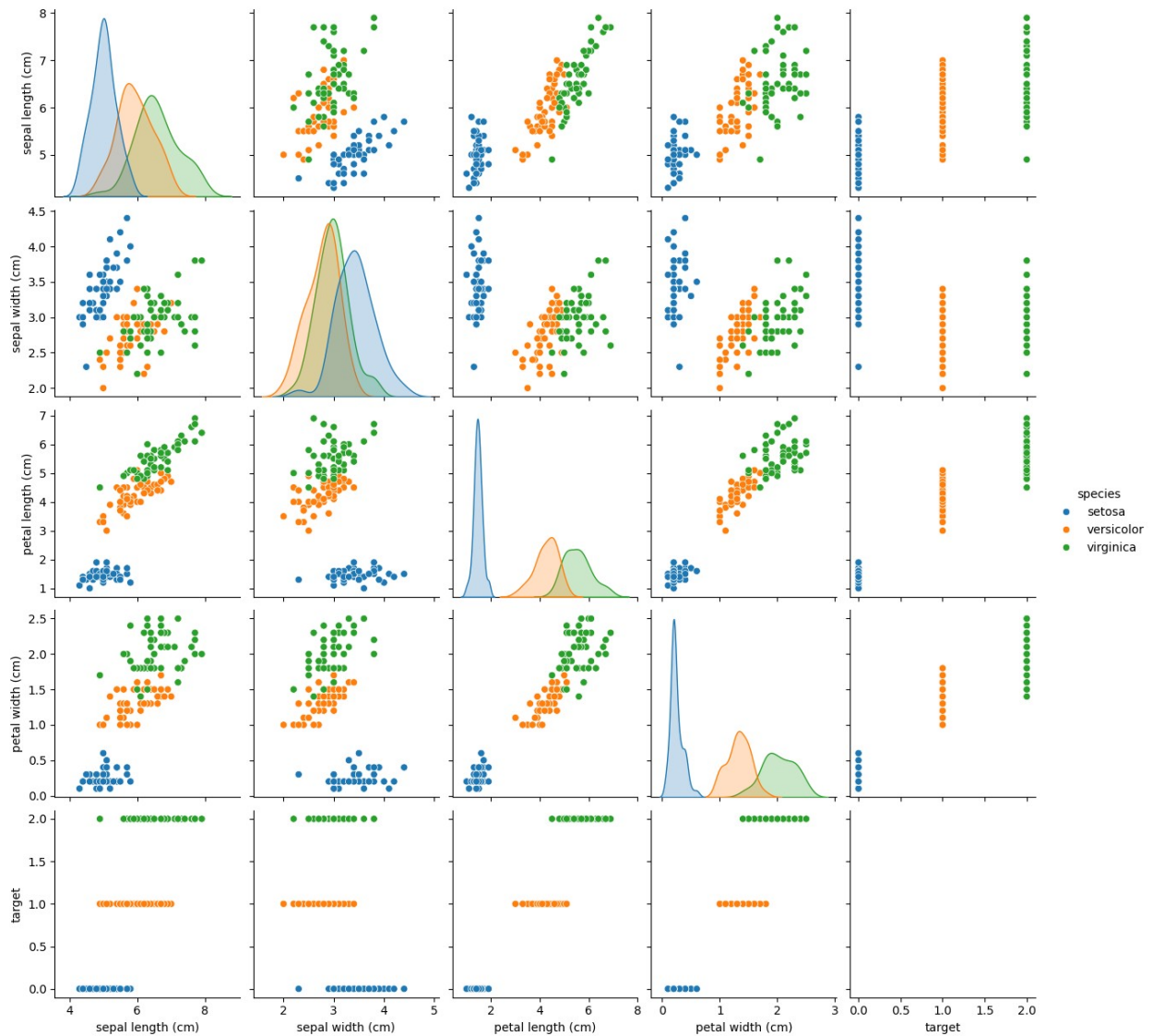
```
# statistical summary of dataset
print("Dataset description:", df.describe())
```

```
Dataset description:      sepal length (cm)  sepal width (cm)  petal
length (cm) \
count      150.000000      150.000000      150.000000
mean         5.843333         3.057333         3.758000
std          0.828066         0.435866         1.765298
min          4.300000         2.000000         1.000000
25%          5.100000         2.800000         1.600000
50%          5.800000         3.000000         4.350000
75%          6.400000         3.300000         5.100000
max          7.900000         4.400000         6.900000

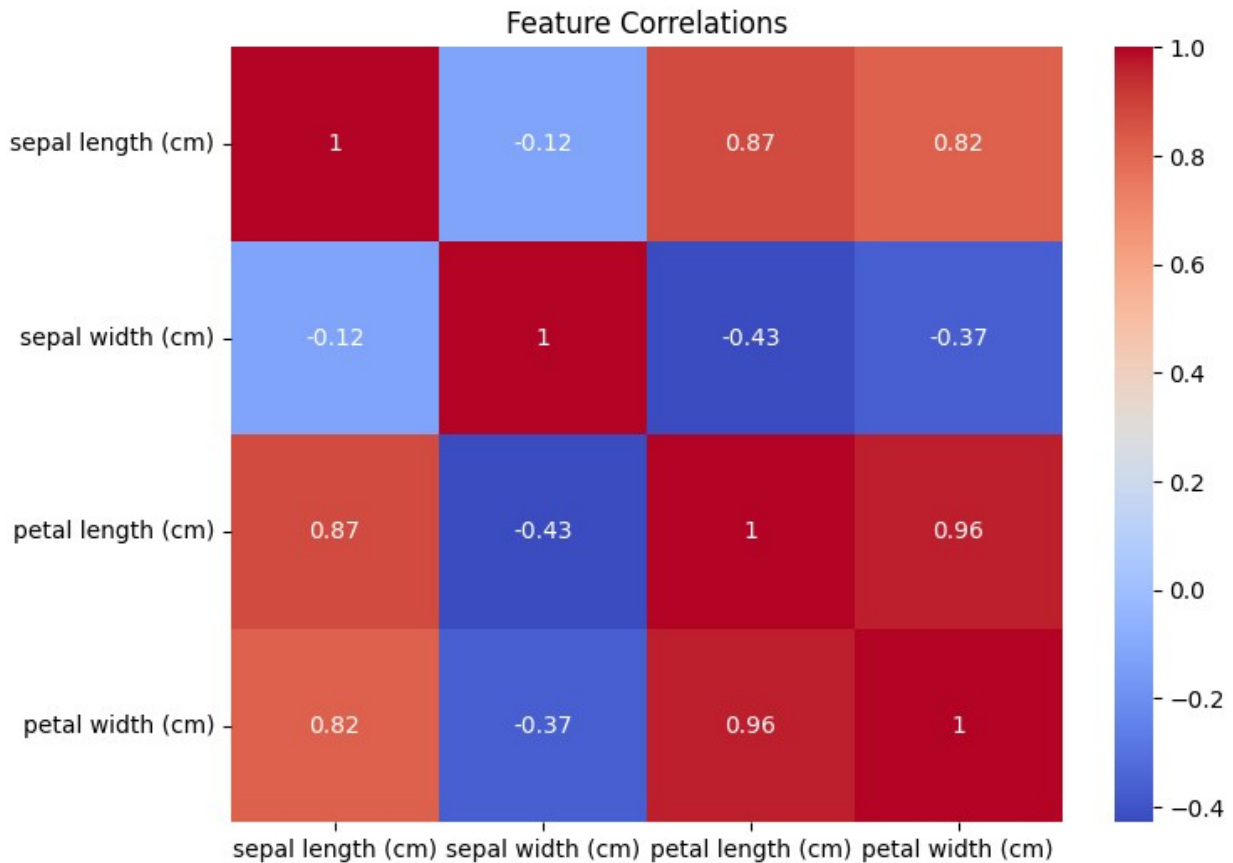
      petal width (cm)  target
count      150.000000  150.000000
mean         1.199333    1.000000
std          0.762238    0.819232
min          0.100000    0.000000
```

25%	0.300000	0.000000
50%	1.300000	1.000000
75%	1.800000	2.000000
max	2.500000	2.000000

```
# plotting a histogram of the features
sns.pairplot(df, hue="species")
plt.show()
```



```
# heatmap of feature correlations
plt.figure(figsize=(8, 6))
sns.heatmap(df[iris.feature_names].corr(), annot=True,
            cmap="coolwarm")
plt.title("Feature Correlations")
plt.show()
```



Task 3: Data Preprocessing

- Split dataset into training and testing sets (e.g., 70% train, 30% test).

```
# splitting the dataset into training and testing sets
X = df[iris.feature_names]
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)
```

Task 4: Build Decision Tree Classifier

- Train the model using DecisionTreeClassifier.

```
# training the decision tree classifier
model = DecisionTreeClassifier(criterion="gini", random_state=42)
model.fit(X_train, y_train)

DecisionTreeClassifier(random_state=42)
```

Task 5: Model Evaluation

- Make predictions on the test data.
- Evaluate using:
 - Accuracy Score

- Confusion Matrix
- Classification Report

```
# making predictions on the test set
```

```
y_pred = model.predict(X_test)
```

```
# evaluating the model with accuracy and classification report
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
classification_rep = classification_report(y_test, y_pred,  
target_names=iris.target_names)
```

```
print("Classification Report:\n", classification_rep)
```

```
Accuracy: 1.0
```

```
Classification Report:
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	19
versicolor	1.00	1.00	1.00	13
virginica	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

```
# confusion matrix
```

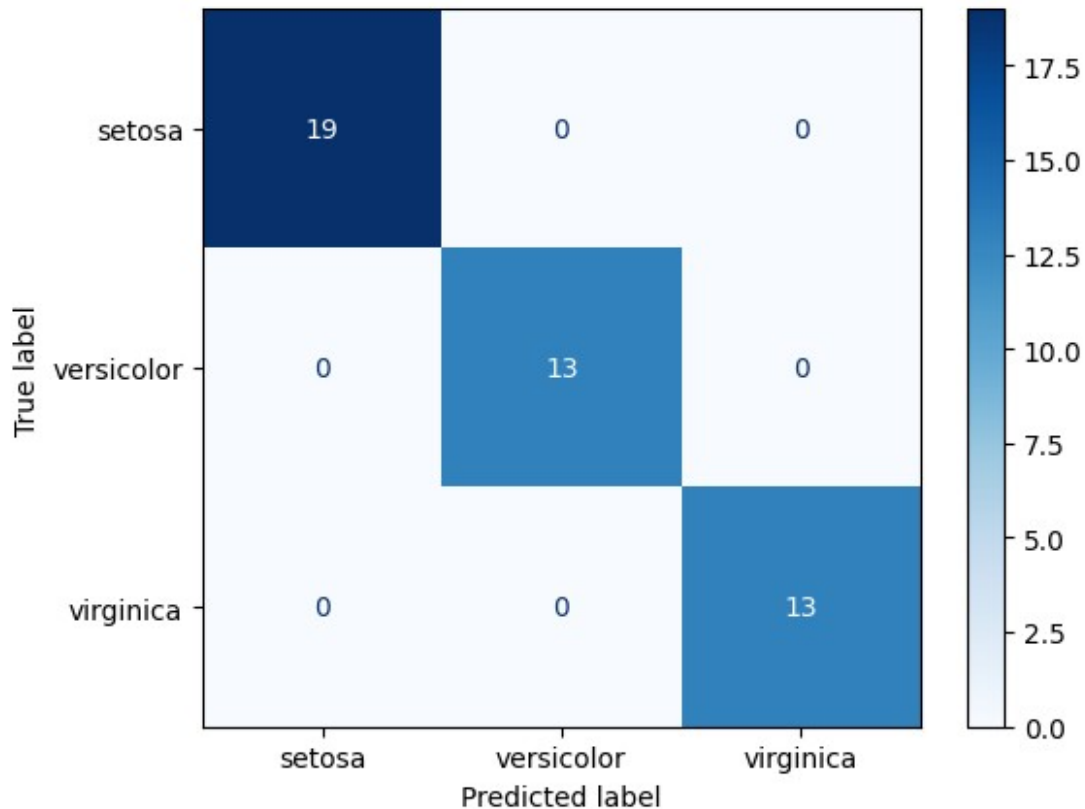
```
cm = confusion_matrix(y_test, y_pred)
```

```
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
```

```
display_labels=iris.target_names)
```

```
disp.plot(cmap=plt.cm.Blues)
```

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at  
0x177a3e4aad0>
```



Task 6: Experimentation

- Change parameters (criterion = "gini" vs "entropy", max_depth, min_samples_split).
- Compare results and observe overfitting vs underfitting.

```
# training and testing accuracy with entropy
treeEntropy = DecisionTreeClassifier(criterion="entropy", max_depth=3,
min_samples_split=2, random_state=42)
treeEntropy.fit(X_train, y_train)
train_accuracy_entropy = treeEntropy.score(X_train, y_train)
test_accuracy_entropy = treeEntropy.score(X_test, y_test)
print("Training Accuracy with Entropy:", train_accuracy_entropy)
print("Testing Accuracy with Entropy:", test_accuracy_entropy)
```

```
Training Accuracy with Entropy: 0.9523809523809523
Testing Accuracy with Entropy: 0.9777777777777777
```

```
# training and testing accuracy with gini index
treeGini = DecisionTreeClassifier(criterion="gini", max_depth=3,
min_samples_split=2, random_state=42)
treeGini.fit(X_train, y_train)
train_accuracy_gini = treeGini.score(X_train, y_train)
test_accuracy_gini = treeGini.score(X_test, y_test)
print("Training Accuracy with Gini:", train_accuracy_gini)
print("Testing Accuracy with Gini:", test_accuracy_gini)
```

Training Accuracy with Gini: 0.9523809523809523
Testing Accuracy with Gini: 1.0

comparing the results of both models

```
results = {  
    "Model": ["Decision Tree (Entropy)", "Decision Tree (Gini)"],  
    "Training Accuracy": [train_accuracy_entropy,  
train_accuracy_gini],  
    "Testing Accuracy": [test_accuracy_entropy, test_accuracy_gini]  
}  
print("Results Comparison:\n", pd.DataFrame(results))
```

Results Comparison:

	Model	Training Accuracy	Testing Accuracy
0	Decision Tree (Entropy)	0.952381	0.977778
1	Decision Tree (Gini)	0.952381	1.000000